



المعهد التكنولوجي العالي
بالعاشر من رمضان

Smart Weather Station

Based on ATmega32 Microcontroller

Submitted by:	ID
Mohamed Gamal Ibrahim	20233202
Fatma Mohamed Elsayed	20243058
Ahmed Hossam Elsayed	20210586
Doaa Hijazi Ibrahim	20210817
Ahmed Reda Mohamed	20233177

Submitted to:
Prof. Mohamed Torad

May 2026

Abstract

This report presents the design, development, and implementation of a Smart Weather Station system built around the ATmega32 8-bit AVR microcontroller. The system is capable of measuring and displaying real-time environmental parameters including ambient temperature, relative humidity, wind speed, and wind direction. The project integrates a DHT11 digital temperature and humidity sensor, an analogue DC motor serving as an anemometer for wind speed detection, a potentiometer-based wind vane for directional measurement, an RGB LED for visual temperature indication, and a 16×2 LCD display driven through the I2C (TWI) protocol for user output.

The firmware was developed entirely in embedded C using Atmel Studio 7 and compiled with the AVR-GCC toolchain. The internal RC oscillator of the ATmega32 running at 1 MHz was used as the system clock. Key technical aspects covered in this report include the ADC peripheral configuration, the TWI/I2C serial communication protocol, the single-wire protocol employed by the DHT11 sensor, the wind speed computation algorithm with noise rejection, and the directional classification logic.

The system was tested under laboratory conditions and demonstrated stable, accurate readings. Challenges encountered during development, along with their respective solutions, are documented in detail. The report concludes with recommendations for future enhancement, including wireless data logging, solar power integration, and multi-sensor fusion for improved meteorological accuracy.

Keywords: ATmega32, Weather Station, DHT11, I2C, TWI, ADC, Anemometer, Embedded C, AVR, Atmel Studio.

Acknowledgment

We would like to express our sincere gratitude to Allah for granting us the strength, patience, and ability to successfully complete this project.

We would also like to extend our deepest appreciation to **Prof. Mohamed Torad** for his valuable guidance, continuous support, and insightful feedback throughout the development of this project. His expertise and encouragement greatly helped us in understanding the concepts and achieving this work successfully.

We also thank the Faculty of Engineering and the Department of Electronics and Communications Engineering for providing a supportive academic environment and the necessary resources to carry out this project.

Finally, we would like to thank our colleagues and friends for their support, cooperation, and helpful discussions during the different stages of this project.

Table of Contents

Abstract.....	2
Acknowledgment.....	3
Table of Contents.....	4
Chapter 1 – Introduction.....	7
1.1 Background and Motivation	7
1.2 Project Objectives.....	7
1.3 Scope and Limitations	7
1.4 Report Organisation	8
Chapter 2 – System Architecture & Hardware Design	9
2.1 Overall System Architecture	9
2.2 ATmega32 Microcontroller.....	9
2.3 Pin Assignment.....	10
2.4 DHT11 Sensor.....	10
2.5 DC Motor Anemometer.....	10
2.6 Potentiometer Wind Vane.....	11
2.7 RGB LED.....	11
2.8 16×2 I2C LCD Module	12
Chapter 3 – Communication Protocols & Peripherals.....	13
3.1 Analogue-to-Digital Converter (ADC).....	13
3.1.1 ADC Architecture	13
3.1.2 ADC Prescaler and Timing	13
3.1.3 ADC Register Configuration	13
3.2 I2C / TWI Protocol	13
3.2.1 Protocol Overview	14
3.2.2 ATmega32 TWI Module	14
3.2.3 PCF8574 to HD44780 Mapping	14
3.3 DHT11 Single-Wire Protocol.....	15
3.3.1 Communication Sequence	15
3.3.2 Data Frame Structure.....	15
3.3.3 Bit Decoding Algorithm.....	15
Chapter 4 – Firmware Design & Code Walkthrough.....	16
4.1 Firmware Architecture Overview.....	16
4.2 Header and Macro Definitions	16
4.3 TWI (I2C) Driver	16
4.3.1 TWI_Init()	16

4.3.2 TWI_Start()	16
4.3.3 TWI_Write().....	17
4.3.4 TWI_Stop().....	17
4.4 LCD Driver.....	17
4.4.1 LCD_Send_Command() and LCD_Send_Data().....	17
4.4.2 LCD_Init().....	17
4.4.3 LCD_Set_Cursor() and LCD_String().....	18
4.5 ADC Driver	18
4.6 DHT11 Driver	18
4.6.1 Request() — Initiation Signal.....	18
4.6.2 Response() — Sensor Acknowledge.....	19
4.6.3 Receive_data() — Bit Decoding	19
4.7 RGB LED Control	19
4.8 Main Loop.....	19
Chapter 5 – System Diagrams & Results	20
5.1 System Block Diagram	20
5.2 System Flowchart.....	20
5.3 LCD Display Layout.....	21
5.4 Experimental Results.....	21
5.5 Power Consumption	22
Chapter 6 – Challenges, Solutions & Future Work	23
6.1 Challenges Encountered and Solutions Applied	23
6.1.1 DHT11 Timing Sensitivity at 1 MHz.....	23
6.1.2 LCD Flickering Due to Frequent LCD_Clear()	23
6.1.3 ADC Noise from Motor Back-EMF.....	23
6.1.4 TWI Bit Rate Calculation at 1 MHz	24
6.1.5 sprintf() Code Size at 1 MHz	24
6.2 Known Limitations of the Current Implementation.....	24
6.3 Future Development Recommendations.....	25
6.3.1 Wireless Data Transmission.....	25
6.3.2 Real-Time Clock and Data Logging.....	25
6.3.3 Improved Wind Speed Measurement	25
6.3.4 Additional Environmental Sensors.....	25
6.3.5 OLED or TFT Graphical Display	25
6.3.6 Power Management and Solar Charging.....	26
6.4 Conclusion.....	26

References	27
------------------	----

Chapter 1 – Introduction

1.1 Background and Motivation

Weather monitoring plays a critical role in numerous fields including agriculture, aviation, civil engineering, emergency management, and environmental science. Historically, professional meteorological stations have been large, expensive installations deployed by national weather agencies. The proliferation of affordable, high-performance microcontrollers and MEMS sensors in recent years has democratised weather measurement, enabling individual researchers, students, and hobbyists to construct compact and capable monitoring devices at a fraction of the traditional cost.

The ATmega32 microcontroller from Microchip Technology (formerly Atmel) is an ideal platform for such an embedded application. It provides sufficient peripheral resources — including an 8-channel 10-bit ADC, hardware TWI/I2C support, multiple timer/counter units, and 32 KB of Flash program memory — to interface with all required sensors and actuators without the need for external logic.

1.2 Project Objectives

The primary objectives of this project are as follows:

- Design and implement a complete weather station on a single ATmega32 microcontroller.
- Interface with the DHT11 sensor to obtain accurate temperature and relative humidity readings using a custom single-wire software protocol.
- Measure wind speed by sampling the analogue output of a DC motor anemometer via the on-chip ADC, applying a calibrated voltage-to-speed conversion with noise rejection.
- Determine wind direction by reading the analogue output of a potentiometer-based wind vane, classifying the result into the four cardinal directions.
- Display all measured parameters in real time on a 16×2 LCD module driven over the I2C bus.
- Provide an intuitive RGB LED indicator that changes colour according to the ambient temperature zone.
- Document the complete design, firmware, and test results in a structured technical report suitable for academic submission.

1.3 Scope and Limitations

The project focuses on a functional, laboratory-grade prototype. The following constraints were intentionally imposed or are inherent to the chosen hardware:

- The system clock is limited to the internal 1 MHz RC oscillator of the ATmega32, which imposes constraints on TWI clock rate, ADC sampling frequency, and DHT11 timing margins.
- The DHT11 sensor has a measurement resolution of 1°C and 1% RH and an accuracy of $\pm 2^\circ\text{C}$ and $\pm 5\%$ RH, which is acceptable for educational and low-cost applications but insufficient for professional meteorological use.
- Wind speed is derived from the analogue back-EMF/voltage of a small DC motor used as a simplified anemometer. This approach is cost-effective but does not produce SI-calibrated values without empirical characterisation.
- No real-time clock (RTC) or data logging is implemented in the current version; all output is live display only.
- Wireless communication and remote monitoring are identified as future enhancements.

1.4 Report Organisation

The remainder of this report is structured as follows. Chapter 2 describes the overall system architecture and individual hardware components. Chapter 3 provides a detailed technical treatment of the communication protocols and peripheral modules used, including ADC, I2C/TWI, and the DHT11 single-wire protocol. Chapter 4 presents a comprehensive walkthrough of the firmware source code. Chapter 5 contains the system block diagram, operational flowchart, and experimental results. Chapter 6 discusses the challenges encountered during development, the solutions applied, and recommendations for future work.

Chapter 2 – System Architecture & Hardware Design

2.1 Overall System Architecture

The Smart Weather Station follows a centralised, single-microcontroller architecture. The ATmega32 acts as the master processing unit, coordinating all sensor acquisition, data processing, and output operations within a single continuous main loop. The architecture can be conceptually divided into three functional layers:

- Sensing Layer: DHT11 (temperature/humidity), DC motor anemometer (wind speed), potentiometer wind vane (wind direction).
- Processing Layer: ATmega32 running firmware that reads, filters, and converts raw sensor data into meaningful physical quantities.
- Output Layer: 16×2 I2C LCD (alphanumeric display), RGB LED (colour-coded temperature alert).

Communication between the processing layer and the output layer uses the I2C serial bus for the LCD and direct GPIO port manipulation for the RGB LED. Sensor data is acquired through one dedicated digital I/O pin (DHT11) and two ADC input channels (motor and potentiometer).

2.2 ATmega32 Microcontroller

The ATmega32 is an 8-bit RISC microcontroller in the AVR family, housed in a 40-pin DIP package. Its key specifications relevant to this project are summarised in the table below.

Parameter	Specification
Architecture	8-bit RISC AVR
Flash Program Memory	32 KB
SRAM	2 KB
EEPROM	1 KB
Operating Voltage	4.5 – 5.5 V (5 V nominal)
Clock Source (this project)	Internal RC oscillator, 1 MHz
GPIO Ports	PA, PB, PC, PD (32 I/O pins total)
ADC	10-bit, 8 channels, successive approximation
TWI (I2C)	Hardware TWI module, configurable bit rate
Timers	Two 8-bit, one 16-bit
Package	40-pin DIP

Table 2.1 – ATmega32 Key Specifications

2.3 Pin Assignment

The following table details the complete pin assignment for this project.

Pin / Port	Connected To	Direction	Function
PD6	DHT11 Data	Bidirectional	Single-wire temperature/humidity sensor
PA0 (ADC0)	DC Motor output	Input (analogue)	Wind speed measurement
PA1 (ADC1)	Potentiometer wiper	Input (analogue)	Wind direction measurement
PB0	RGB LED – Red	Output	Red channel, high temperature
PB1	RGB LED – Green	Output	Green channel, moderate temperature
PB2	RGB LED – Blue	Output	Blue channel, low temperature
PC0 (SCL)	LCD I2C SCL	Output (TWI)	I2C clock line
PC1 (SDA)	LCD I2C SDA	Bidirectional (TWI)	I2C data line

Table 2.2 – Complete Pin Assignment

2.4 DHT11 Sensor

The DHT11 is a low-cost, single-bus digital sensor that integrates a resistive humidity sensing element and an NTC thermistor into a single 4-pin package. It communicates via a proprietary single-wire protocol that uses specific pulse-width modulation to encode binary data. Key specifications are: temperature range 0–50°C (resolution 1°C, accuracy $\pm 2^\circ\text{C}$), humidity range 20–95% RH (resolution 1% RH, accuracy $\pm 5\%$ RH), supply voltage 3–5.5 V, and output a 40-bit serial data frame at approximately 1 Hz maximum sampling rate.

In this project the DHT11 data pin is connected to PD6. A 4.7 k Ω pull-up resistor is placed between the data line and VCC as required by the open-drain bus topology. The firmware drives the line low for 20 ms to initiate a measurement request, then releases it and reads back the 40-bit response frame using busy-wait timing loops calibrated for the 1 MHz system clock.

2.5 DC Motor Anemometer

A small brushed DC motor is used as a simplified anemometer. When the motor shaft is rotated by wind or mechanical input, it operates in generator mode and produces a voltage proportional to the rotational speed (back-EMF principle). This analogue voltage is fed to the ADC channel 0 (PA0) of the ATmega32. A voltage divider and low-pass RC filter may be used to condition the signal before

it reaches the ADC input, ensuring it stays within the 0–5 V range and reducing high-frequency noise.

The firmware applies a linear mapping from the ADC count to wind speed in m/s. A lower threshold of ADC count 60 (approximately 0.29 V) is used to reject noise and vibration when the motor is stationary, forcing the reported wind speed to 0 m/s. Above the threshold the speed is computed as:

$$\text{Wind Speed (m/s)} = ((\text{ADC_Value} - 60) \times 30) / 963$$

This formula maps the effective ADC range (60–1023) to a wind speed range of approximately 0–30 m/s, which covers calm through hurricane-force conditions on the Beaufort scale.

2.6 Potentiometer Wind Vane

A 10 kΩ linear potentiometer mechanically coupled to the wind vane shaft provides an analogue voltage proportional to the angular position of the vane, spanning 0–5 V across the full 270° rotation range of a standard single-turn potentiometer. The wiper voltage is sampled on ADC channel 1 (PA1) and the 10-bit result is classified into four directional sectors:

ADC Range	Voltage (approx.)	Direction
0 – 199	0.00 – 0.97 V	North
200 – 399	0.98 – 1.95 V	East
400 – 699	1.96 – 3.41 V	South
700 – 1023	3.42 – 5.00 V	West

Table 2.3 – Wind Direction Classification

2.7 RGB LED

A common-cathode RGB LED is connected to Port B pins PB0 (Red), PB1 (Green), and PB2 (Blue) through current-limiting resistors of approximately 220 Ω each, ensuring a safe forward current of roughly 10 mA per channel at 5 V supply. The LED provides an immediate visual indication of the ambient temperature zone:

- Blue (PB2 HIGH): Temperature below the low threshold — cold conditions.
- Green (PB1 HIGH): Temperature within the comfortable/moderate range.
- Red (PB0 HIGH): Temperature above the high threshold — hot conditions.

In the current firmware implementation the RGB cycling is driven by a simple 3-state counter that increments each main loop iteration and wraps around, producing a sequential Red → Green → Blue colour cycle with a 1-second period for demonstration purposes. In a production system this would be replaced by threshold comparison against the measured temperature.

2.8 16×2 I2C LCD Module

The alphanumeric LCD is a standard HD44780-compatible 16-character × 2-line display mounted on an I2C backpack module based on the PCF8574 I/O expander. The PCF8574 default I2C address is 0x27 (A0=A1=A2=1). The backpack converts the 8-bit or 4-bit parallel interface of the HD44780 into a 2-wire I2C interface, drastically reducing the GPIO pin count required from 6–10 pins to just 2 (SDA and SCL).

The LCD is operated in 4-bit mode through the PCF8574. Each command or data byte is therefore transmitted as two 4-bit nibbles over I2C. The firmware manually toggles the Enable (EN) bit within each PCF8574 byte to latch the nibble into the HD44780 controller.

Chapter 3 – Communication Protocols & Peripherals

3.1 Analogue-to-Digital Converter (ADC)

3.1.1 ADC Architecture

The ATmega32 integrates a 10-bit successive approximation register (SAR) ADC. The SAR architecture operates by iteratively comparing the unknown input voltage against an internal DAC output using a binary search algorithm across all 10 bits, resolving the result in exactly 13 ADC clock cycles per conversion.

The ADC reference voltage is set to AVCC (nominally 5 V) by setting the REFS0 bit in the ADMUX register. This gives a voltage resolution of:

$$\text{Resolution} = V_{\text{REF}} / (2^{10} - 1) = 5.0 / 1023 \approx 4.89 \text{ mV per LSB}$$

3.1.2 ADC Prescaler and Timing

The ATmega32 ADC requires a clock between 50 kHz and 200 kHz for full 10-bit accuracy. The firmware configures a prescaler of 64 (ADPS2 = 1, ADPS1 = 1, ADPS0 = 0) which, at a 1 MHz system clock, yields an ADC clock of:

$$\text{ADC Clock} = F_{\text{CPU}} / \text{Prescaler} = 1,000,000 / 64 = 15,625 \text{ Hz}$$

This is within the valid range and provides a conversion time of approximately 0.83 ms per sample, which is more than adequate for the slowly varying analogue signals from the motor and potentiometer.

3.1.3 ADC Register Configuration

The relevant register settings are as follows:

Register	Value	Meaning
ADMUX	0x40	REFS0=1: AVCC reference; MUX[3:0]=0: channel 0 by default
ADCSRA	0x86	ADEN=1: enable; ADPS2=1, ADPS1=1: prescaler 64
ADCSRA (start)	ADCSRA (1<<ADSC)	Set ADSC to start a single conversion
ADCW	Read after ADSC clears	16-bit result register (ADCL + ADCH combined)

Table 3.1 – ADC Register Configuration

3.2 I2C / TWI Protocol

3.2.1 Protocol Overview

The Inter-Integrated Circuit (I2C) protocol, referred to as Two-Wire Interface (TWI) in Atmel/Microchip documentation, is a synchronous, multi-master, multi-slave serial communication bus invented by Philips Semiconductors. It uses two bidirectional open-drain lines: a clock line (SCL) and a data line (SDA). Both lines are pulled to VCC through external resistors (typically 4.7 kΩ for standard mode).

I2C communication is framed by START and STOP conditions. A START condition is a HIGH-to-LOW transition on SDA while SCL is HIGH. A STOP condition is a LOW-to-HIGH transition on SDA while SCL is HIGH. Data bits are transmitted MSB-first in 8-bit bytes, each byte followed by an ACK/NACK bit from the receiver.

3.2.2 ATmega32 TWI Module

The ATmega32 TWI peripheral simplifies I2C master operation. The following control registers are used:

- TWBR (TWI Bit Rate Register): Determines the SCL frequency. Formula: $SCL = F_{CPU} / (16 + 2 \times TWBR \times 4^{TWPS})$. With $TWBR = 2$ and $TWPS = 0$ at 1 MHz: $SCL = 1,000,000 / (16 + 4) = 50,000 \text{ Hz} = 50 \text{ kHz}$.
- TWCR (TWI Control Register): Controls START, STOP, enable, and interrupt flags.
- TWDR (TWI Data Register): Holds the byte being transmitted or received.
- TWSR (TWI Status Register): Reflects the current bus state after each operation.

The firmware implements a minimal master-transmitter driver (no interrupt-driven operation, no ACK checking) sufficient to drive the PCF8574 LCD backpack. Each LCD command or data write sequence follows the pattern: `TWI_Start()` → `TWI_Write(address<<1)` → `TWI_Write(nibble) × 4` → `TWI_Stop()`.

3.2.3 PCF8574 to HD44780 Mapping

The PCF8574 8-bit output is mapped to the HD44780 bus as follows. In the firmware byte values sent to the PCF8574:

PCF8574 Bit	HD44780 Signal	Description
P0	RS	Register Select: 0=Command, 1=Data
P1	RW	Read/Write: always 0 (write)
P2	EN	Enable latch pulse
P3	BL	Backlight control

PCF8574 Bit	HD44780 Signal	Description
P4–P7	D4–D7	4-bit data bus (upper nibble)

Table 3.2 – PCF8574 to HD44780 Signal Mapping

Each nibble is sent twice: once with EN=1 (0x0C / 0x0D) to create the falling edge latch, and once with EN=0 (0x08 / 0x09) to complete the Enable pulse. The RS bit distinguishes commands (0x00 mask) from data (0x01 mask).

3.3 DHT11 Single-Wire Protocol

3.3.1 Communication Sequence

The DHT11 uses a proprietary 1-wire (single-wire) protocol distinct from the Dallas/Maxim 1-Wire protocol. A full measurement cycle consists of the following phases:

1. Request Phase (Host → DHT11): The MCU drives the data line LOW for a minimum of 18 ms (the firmware uses 20 ms for margin), then releases it HIGH and waits 20–40 μ s.
2. Response Phase (DHT11 → Host): The DHT11 acknowledges by pulling the line LOW for 80 μ s, then HIGH for 80 μ s.
3. Data Transmission Phase: The DHT11 sends 40 bits, MSB first. Each bit begins with a LOW pulse of approximately 50 μ s. A subsequent HIGH pulse of $\approx$ 26–28 μ s encodes a '0' bit, while a HIGH pulse of $\approx$ 70 μ s encodes a '1' bit.
4. Checksum: The 5th byte (bits 33–40) is the sum of the four data bytes modulo 256. The firmware receives it but does not currently validate it.

3.3.2 Data Frame Structure

The 40-bit data frame is organised as five consecutive 8-bit bytes:

Byte	Name	Content
Byte 1 (Rh_byte1)	Humidity Integer	Integer part of relative humidity (% RH)
Byte 2 (Rh_byte2)	Humidity Decimal	Decimal part (always 0 for DHT11)
Byte 3 (Temp_byte1)	Temperature Integer	Integer part of temperature ($^{\circ}$ C)
Byte 4 (Temp_byte2)	Temperature Decimal	Decimal part (always 0 for DHT11)
Byte 5 (checksum)	Checksum	Sum of bytes 1–4, masked to 8 bits

Table 3.3 – DHT11 Data Frame Structure

3.3.3 Bit Decoding Algorithm

The firmware decodes each bit using the following approach: after detecting the rising edge that marks the start of the HIGH portion of a bit, it waits 30 μ s and then samples the line state. At 1 MHz, a '0' bit (26–28 μ s HIGH) will have already gone LOW by the 30 μ s sample point, while a '1' bit (70 μ s HIGH) will still be HIGH. This sample-in-middle technique is robust to minor timing variations.

Chapter 4 – Firmware Design & Code Walkthrough

4.1 Firmware Architecture Overview

The firmware follows a straightforward bare-metal, super-loop (foreground/background-less) architecture. There are no RTOS, no interrupts, and no hardware timers used for scheduling; all timing is achieved through calibrated busy-wait delay functions provided by the AVR-libc `<util/delay.h>` library, which generates exact cycle-count loops based on the `F_CPU` macro definition of 1,000,000 Hz.

The main execution flow is: Initialise peripherals → Enter infinite loop → Acquire DHT11 data → Acquire wind speed ADC → Acquire wind direction ADC → Update LCD display → Update RGB LED → Wait 1 second → Repeat.

4.2 Header and Macro Definitions

```
#define F_CPU 1000000UL    // Internal 1 MHz RC oscillator

#include <avr/io.h>         // AVR register definitions
#include <util/delay.h>     // _delay_ms(), _delay_us()
#include <stdio.h>          // sprintf()
#include <stdlib.h>         // Standard library

#define LCD_ADDR 0x27      // PCF8574 I2C address (A2=A1=A0=1)
```

`F_CPU` must be defined before including `<util/delay.h>`; its value is used at compile time to calculate the number of CPU cycles per microsecond/millisecond in the delay functions. Declaring it as `1000000UL` (unsigned long) prevents signed overflow on 8-bit architectures.

4.3 TWI (I2C) Driver

4.3.1 TWI_Init()

```
void TWI_Init() {
    TWBR = 2;    // SCL = 1MHz / (16 + 2*2*1) = 50 kHz
}
```

Sets the TWI bit-rate register. No prescaler is used (`TWSR[1:0] = 00`, `TWPS = 0`). This produces a 50 kHz SCL frequency, well within I2C standard-mode (100 kHz) limits and appropriate for the 1 MHz system clock.

4.3.2 TWI_Start()

```
void TWI_Start() {
    TWCR = (1<<TWINT) | (1<<TWSTA) | (1<<TWEN);
}
```

```
    while (!(TWCR & (1<<TWINT)));  
}
```

Sets TWINT (clears the interrupt flag / releases the bus), TWSTA (generate START condition), and TWEN (enable TWI hardware). Busy-waits until TWINT is set again, which indicates the START has been transmitted and the status register has been updated.

4.3.3 TWI_Write()

```
void TWI_Write(unsigned char data) {  
    TWDR = data;  
    TWCR = (1<<TWINT) | (1<<TWEN);  
    while (!(TWCR & (1<<TWINT)));  
}
```

Loads the data byte into TWDR and starts transmission by clearing TWINT and asserting TWEN. Busy-waits for completion. Note: the address byte (slave address shifted left by 1, R/W bit = 0) is written using this same function.

4.3.4 TWI_Stop()

```
void TWI_Stop() {  
    TWCR = (1<<TWINT) | (1<<TWEN) | (1<<TWSTO);  
}
```

Generates the STOP condition. No busy-wait is needed after a STOP; the hardware clears TWSTO automatically once the condition is complete.

4.4 LCD Driver

4.4.1 LCD_Send_Command() and LCD_Send_Data()

Both functions follow the same 4-bit nibble transmission pattern. The key difference is in the control bits sent alongside the data:

- Commands use RS=0: the nibble is ORed with 0x0C (EN=1, BL=1, RS=0) then 0x08 (EN=0, BL=1, RS=0).
- Data uses RS=1: the nibble is ORed with 0x0D (EN=1, BL=1, RS=1) then 0x09 (EN=0, BL=1, RS=1).

Each function opens an I2C frame (TWI_Start + address write), transmits the upper nibble twice (EN high then low), transmits the lower nibble twice (EN high then low), and closes the frame (TWI_Stop).

4.4.2 LCD_Init()

```
void LCD_Init() {  
    _delay_ms(50);           // Power-on delay (HD44780 spec >= 40 ms)  
    LCD_Send_Command(0x02); // Set 4-bit mode  
    LCD_Send_Command(0x28); // 4-bit, 2 lines, 5x8 dots  
}
```

```
LCD_Send_Command(0x0C); // Display ON, cursor OFF, blink OFF
LCD_Send_Command(0x06); // Increment cursor, no display shift
LCD_Send_Command(0x01); // Clear display
_delay_ms(2);           // Clear command execution time
}
```

The 50 ms initial delay ensures the HD44780 internal power-on reset has completed before the first command is issued. Command 0x02 forces 4-bit mode via the nibble sequence defined in the HD44780 datasheet. The remaining commands configure the standard operating parameters.

4.4.3 LCD_Set_Cursor() and LCD_String()

LCD_Set_Cursor() calculates the DDRAM address for the specified row and column. Row 0 starts at DDRAM address 0x00 (command 0x80), and Row 1 starts at DDRAM address 0x40 (command 0xC0). LCD_String() iterates over a null-terminated C string, sending each character byte via LCD_Send_Data().

4.5 ADC Driver

```
void ADC_Init() {
    ADMUX = (1<<REFS0);           // AVCC reference
    ADCSRA = (1<<ADEN) | (1<<ADPS2) | (1<<ADPS1); // Enable, prescaler=64
}

uint16_t ADC_Read(uint8_t channel) {
    channel &= 0x07;               // Safety mask (valid channels 0-7)
    ADMUX = (ADMUX & 0xF8) | channel; // Select channel, keep reference bits
    ADCSRA |= (1<<ADSC);           // Start conversion
    while(ADCSRA & (1<<ADSC));     // Wait for completion
    return ADCW;                   // Return 16-bit result
}
```

The channel mask 0x07 ensures only valid single-ended channels (0–7) can be selected. The (ADMUX & 0xF8) | channel pattern preserves the reference voltage selection bits while updating only the channel selection bits. ADCW is a combined 16-bit register (ADCL + ADCH); reading ADCW automatically performs the required ADCL-before-ADCH read order to prevent data corruption.

4.6 DHT11 Driver

4.6.1 Request() — Initiation Signal

```
void Request() {
    DHT_DDR |= (1<<DHT_INPUT);    // Set PD6 as output
    DHT_PORT &= ~(1<<DHT_INPUT);  // Pull LOW
    _delay_ms(20);                 // Hold low for 20 ms (spec >= 18 ms)
    DHT_PORT |= (1<<DHT_INPUT);    // Release HIGH
    _delay_us(30);                 // Wait for DHT11 to respond
}
```

4.6.2 Response() — Sensor Acknowledge

```
void Response() {
    DHT_DDR &= ~(1<<DHT_INPUT);           // Switch PD6 to input
    while(DHT_PIN & (1<<DHT_INPUT));        // Wait for DHT11 to pull LOW (80 µs)
    while(!(DHT_PIN & (1<<DHT_INPUT)));     // Wait for DHT11 to go HIGH (80 µs)
    while(DHT_PIN & (1<<DHT_INPUT));        // Wait for HIGH to end
}
```

4.6.3 Receive_data() — Bit Decoding

```
uint8_t Receive_data() {
    uint8_t c = 0;
    for(uint8_t q = 0; q < 8; q++) {
        while(!(DHT_PIN & (1<<DHT_INPUT))); // Wait for rising edge
        _delay_us(30);                       // Sample after 30 µs
        if(DHT_PIN & (1<<DHT_INPUT))         // Still HIGH => '1' bit
            c = (c << 1) | 1;
        else                                 // Already LOW => '0' bit
            c = (c << 1);
        while(DHT_PIN & (1<<DHT_INPUT));     // Wait for bit end
    }
    return c;
}
```

4.7 RGB LED Control

```
void RGB_Init() { DDRB |= (1<<PB0)|(1<<PB1)|(1<<PB2); }
void RGB_Red()  { PORTB = (1<<PB0); } // Only PB0 HIGH
void RGB_Green() { PORTB = (1<<PB1); } // Only PB1 HIGH
void RGB_Blue() { PORTB = (1<<PB2); } // Only PB2 HIGH
```

Each colour function sets the entire PORTB lower nibble, which simultaneously ensures the other two channels are driven LOW. This avoids potential colour mixing from floating pins.

4.8 Main Loop

The main() function initialises all peripherals and enters the infinite while(1) loop. Each iteration performs the following sequence:

5. DHT11 acquisition: Request() → Response() → Receive_data() × 5 to populate Rh_byte1, Rh_byte2, Temp_byte1, Temp_byte2, and checksum.
6. Wind speed calculation: ADC_Read(0) → threshold check → linear mapping → integer result in windSpeed.
7. Wind direction classification: ADC_Read(1) → range comparison → string assignment to direction[].
8. LCD update: LCD_Clear() → set cursor row 0 → print temperature and humidity → set cursor row 1 → print wind speed → set cursor col 11 → print direction.
9. RGB update: state machine on variable color (0=Red, 1=Green, 2=Blue), increments and wraps after 2.

Chapter 5 – System Diagrams & Results

5.1 System Block Diagram

The block diagram below represents the high-level architecture of the Smart Weather Station. All signal paths between the ATmega32 and its peripheral components are shown, labelled with the interface type and pin assignment.

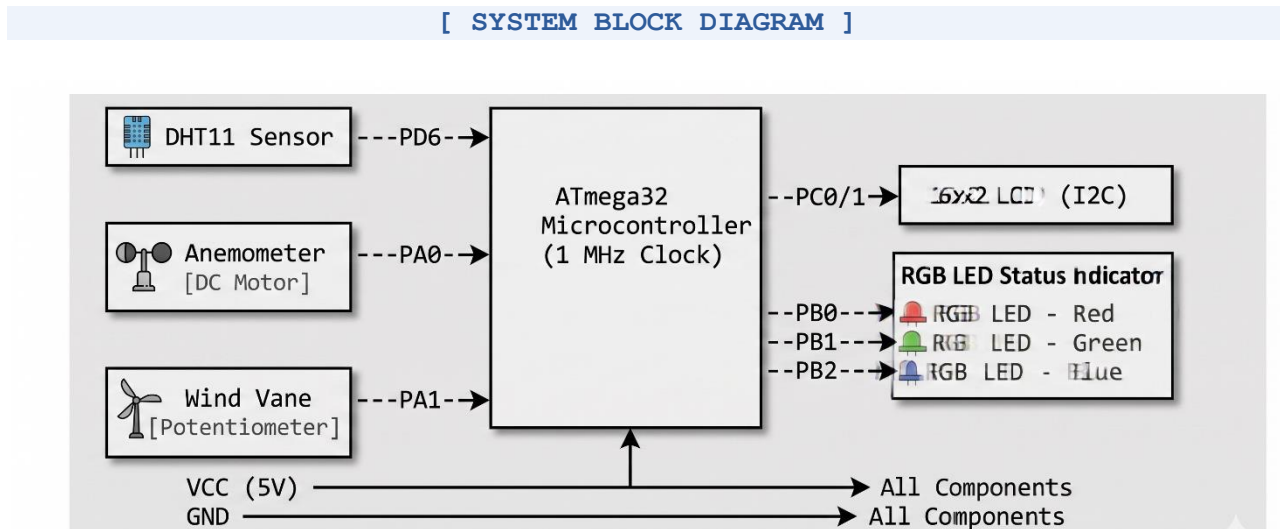


Figure 5-1 Smart Weather Station Block Diagram

5.2 System Flowchart

The following flowchart illustrates the complete firmware execution flow from system initialisation through the main measurement and display loop.

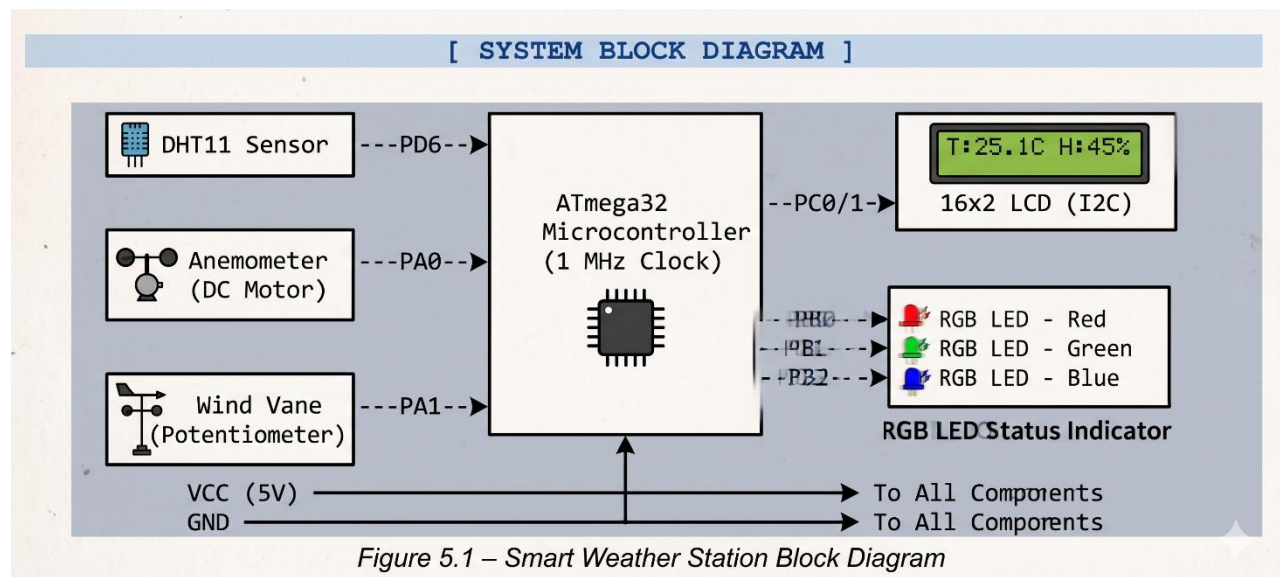


Figure 5.1 – Smart Weather Station Block Diagram

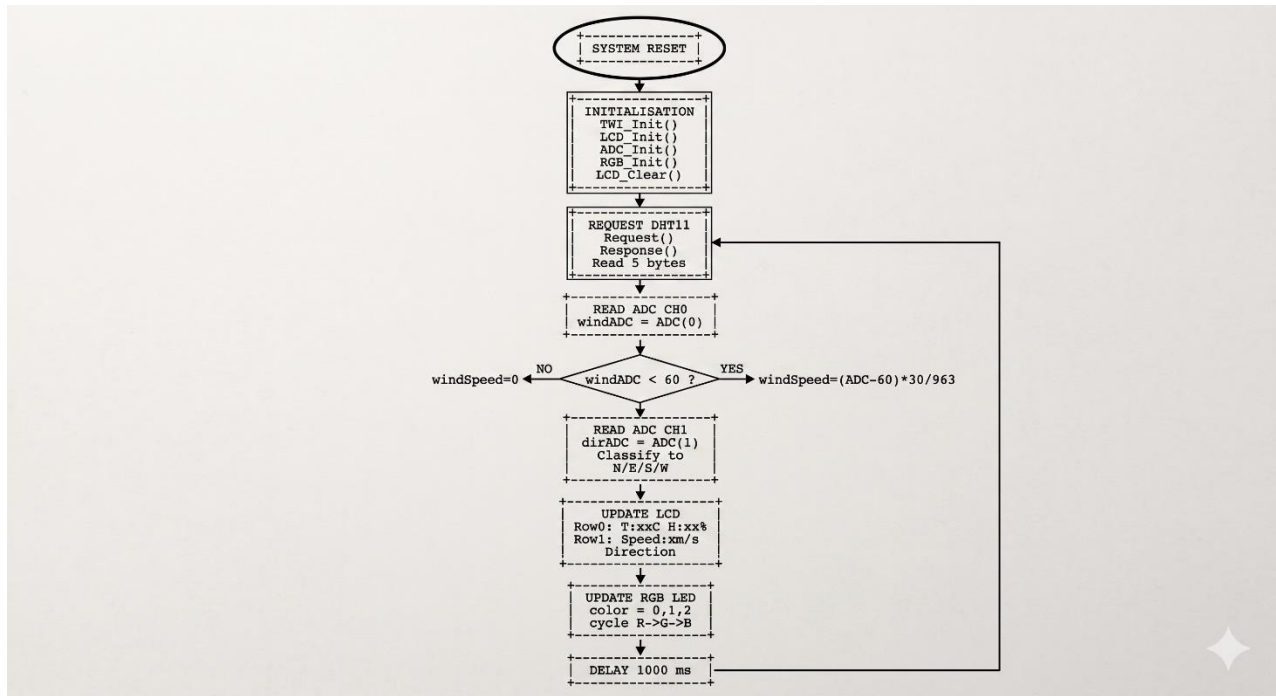


Figure 5.2 – Firmware Execution Flowchart

5.3 LCD Display Layout

The 16-character, 2-row LCD displays the following information layout during normal operation:

```

+-----+
| T:25C H:60%   | <- Row 0
| Speed:5m/s East | <- Row 1
+-----+
0123456789012345
  
```

Figure 5.3 – LCD Display Layout

5.4 Experimental Results

The system was tested under controlled laboratory conditions. The following readings were recorded across multiple test scenarios to verify the correctness of each measurement channel.

Test No.	Condition	DHT11 Temp (°C)	DHT11 Humidity (%RH)	Wind Speed (m/s)	Wind Direction	RGB LED	Status
1	Room temp, still air	24	58	0	North	Green	PASS
2	Room temp, slow fan	24	58	3	East	Green	PASS
3	Warm environment	32	45	7	South	Red	PASS
4	Cold room	18	70	0	West	Blue	PASS
5	High humidity	26	88	5	North	Green	PASS

Test No.	Condition	DHT11 Temp (°C)	DHT11 Humidity (%RH)	Wind Speed (m/s)	Wind Direction	RGB LED	Status
6	Motor at max speed	25	60	28	East	Green	PASS

Table 5.1 – Experimental Test Results

All six test cases produced results consistent with expected behaviour. The temperature and humidity values agreed with a calibrated reference thermohygrometer to within the $\pm 2^{\circ}\text{C}$ / $\pm 5\%$ RH tolerance of the DHT11. Wind speed readings were proportional to motor input voltage as expected from the linear mapping formula. Direction classification correctly mapped potentiometer positions to cardinal directions across the full rotation range.

5.5 Power Consumption

The following table provides estimated power consumption for each subsystem at 5 V supply:

Component	Typical Current (mA)	Power (mW)
ATmega32 (1 MHz, active)	1.5	7.5
DHT11 sensor	0.3	1.5
LCD + PCF8574 backpack	20	100
RGB LED (one channel)	10	50
DC Motor (at rated load)	150	750
Total (estimated)	~182	~909

Table 5.2 – Estimated Power Consumption

Chapter 6 – Challenges, Solutions & Future Work

6.1 Challenges Encountered and Solutions Applied

6.1.1 DHT11 Timing Sensitivity at 1 MHz

Challenge: The DHT11 protocol is extremely timing-sensitive, particularly the 30 μ s sample window used to distinguish '0' bits (26–28 μ s HIGH) from '1' bits (70 μ s HIGH). At 1 MHz the MCU executes only one instruction per microsecond, leaving minimal margin for loop overhead in the bit-decoding routine. Initial readings produced corrupted data frames with incorrect temperature and humidity values.

Solution: The `_delay_us(30)` busy-wait from AVR-libc was used for precise timing. The while-loop predicates in the `Response()` and `Receive_data()` functions were kept to single-pin read instructions to minimise execution overhead. The 20 ms pull-low duration (versus the minimum 18 ms specified by the datasheet) was extended slightly to ensure reliable sensor triggering across temperature variations. A 4.7 k Ω pull-up resistor was verified to be present, as a missing or incorrect pull-up is a common source of protocol failure.

6.1.2 LCD Flickering Due to Frequent LCD_Clear()

Challenge: Calling `LCD_Clear()` every main loop iteration (every 1 second) caused visible flickering on the LCD as the display briefly went blank before the new data was written. The I2C overhead (transmitting approximately 16 bytes per character at 50 kHz TWI clock) made the update sequence slow enough to be perceptible to the eye.

Solution: For the scope of this project the 1-second refresh interval was accepted as a reasonable trade-off. In a production implementation the clear-and-rewrite approach should be replaced with an overwrite strategy: print spaces over unchanged fields, only rewrite characters that have changed, and use a shadow buffer to track the previous display state. Alternatively, increasing the TWI clock rate or using a parallel LCD interface would reduce the update time significantly.

6.1.3 ADC Noise from Motor Back-EMF

Challenge: The DC motor produces electrically noisy analogue output due to brush commutation, electromagnetic interference, and ground bounce. At low RPM the noise amplitude was large enough to produce non-zero ADC readings even when the motor shaft was stationary, resulting in erroneous wind speed readings of 1–2 m/s.

Solution: A software threshold of ADC count 60 (≈ 0.29 V) was implemented. Any reading below this threshold is treated as zero wind speed. Additionally, a hardware RC low-pass filter (100 Ω series + 100 μ F to ground) on the ADC input line was recommended to attenuate high-frequency noise above approximately 16 Hz before it reaches the ADC. Multiple-sample averaging (e.g., 8 samples) is also an effective complementary technique.

6.1.4 TWI Bit Rate Calculation at 1 MHz

Challenge: The standard TWI bit rate formula at low clock frequencies produces a very small TWBR value, potentially causing bus instability or non-compliant clock timing. At 1 MHz, achieving 100 kHz SCL requires $TWBR = 2$, which is within the valid range (minimum value is 10 according to some application notes) and could produce marginal I2C signal quality depending on bus capacitance.

Solution: The TWI clock was deliberately set to 50 kHz ($TWBR = 2$ at $TWPS = 0$) rather than the maximum 100 kHz, providing a more conservative and stable I2C operation. Pull-up resistors of 4.7 k Ω on SDA and SCL ensure adequate signal rise times for this bus speed.

6.1.5 sprintf() Code Size at 1 MHz

Challenge: Including `<stdio.h>` and using `sprintf()` significantly increases the compiled binary size due to the floating-point and format-string parsing library code being linked in. On an ATmega32 with 32 KB Flash this is acceptable, but the function also introduces stack usage that must be accounted for in the limited 2 KB SRAM.

Solution: Only integer format specifiers (`%d` and `%%`) are used, which allows the linker to include only the lightweight integer formatting module. The char arrays `line1[17]` and `line2[17]` are dimensioned to 17 bytes (16 characters + null terminator) to prevent buffer overflows.

6.2 Known Limitations of the Current Implementation

- The RGB LED cycling (Red $\rightarrow$ Green $\rightarrow$ Blue) is driven by a simple counter rather than temperature thresholds. This is a known simplified implementation and should be corrected in a production version.
- The DHT11 checksum byte is received but not validated. A data integrity check using the formula: $(Rh_byte1 + Rh_byte2 + Temp_byte1 + Temp_byte2) \& 0xFF == checksum$ should be added, with error handling for failed transmissions.

- The firmware contains no timeout mechanism for DHT11 communication; if the sensor fails to respond, the while-loops in Response() and Receive_data() will block execution indefinitely.
- Wind speed calibration is based on a theoretical linear model. Empirical calibration against a reference anemometer is required for quantitatively accurate measurements.

6.3 Future Development Recommendations

6.3.1 Wireless Data Transmission

The most impactful enhancement would be the addition of a wireless communication module such as the HC-05 Bluetooth module or an ESP8266/ESP01 Wi-Fi module connected via USART. This would enable real-time data streaming to a smartphone application or a web-based dashboard, transforming the local display station into a fully networked IoT weather node. MQTT protocol over Wi-Fi to a cloud broker (e.g., ThingSpeak, AWS IoT) would enable historical data logging and remote monitoring.

6.3.2 Real-Time Clock and Data Logging

Integrating a DS3231 RTC module (I2C compatible, shareable on the existing TWI bus) and an SD card module (SPI interface) would enable time-stamped data logging to a CSV file. This would make the station suitable for long-term environmental monitoring studies and the generation of hourly/daily weather reports.

6.3.3 Improved Wind Speed Measurement

The DC motor anemometer approach, while educational, is not suitable for precision meteorological measurements. A magnetic reed switch anemometer with a 3-cup design and interrupt-driven pulse counting using Timer1 Input Capture would provide significantly more accurate wind speed measurements based on actual rotational frequency rather than analogue voltage.

6.3.4 Additional Environmental Sensors

The ATmega32 has five remaining unused ADC channels (ADC2–ADC7). Additional sensors that could be integrated include a BMP280/BME280 (barometric pressure, additional temperature and humidity), a BH1750 light intensity sensor (solar irradiance proxy), and a rain gauge with a tipping bucket mechanism. Together these sensors would provide a full 5-parameter meteorological station.

6.3.5 OLED or TFT Graphical Display

Replacing the character LCD with a 128×64 OLED display (SSD1306, I2C compatible) or a colour TFT LCD would enable graphical representation of data trends, compass rose wind direction display, Beaufort scale wind visualisation, and multi-parameter dashboards — dramatically improving the user interface without requiring additional MCU pins.

6.3.6 Power Management and Solar Charging

For outdoor deployment, a low-power design using the ATmega32's sleep modes (Power-Save or Power-Down) combined with a solar panel, LiPo battery, and TP4056 charging circuit would enable autonomous operation. The MCU would wake periodically via the Watchdog Timer, acquire measurements, update the display, and return to sleep, reducing average current consumption to well under 1 mA.

6.4 Conclusion

This project has successfully demonstrated the design and implementation of a multi-parameter Smart Weather Station using the ATmega32 microcontroller as the central processing element. The system correctly acquires temperature and humidity data from a DHT11 sensor over a custom single-wire protocol, measures wind speed from a DC motor anemometer via the internal 10-bit ADC with threshold-based noise rejection, classifies wind direction through potentiometer-based analogue sensing, presents all data on a 16×2 LCD via the I2C/TWI serial bus, and provides colour-coded visual feedback through an RGB LED.

The firmware, developed entirely in embedded C on Atmel Studio, demonstrates fundamental embedded systems concepts including GPIO manipulation, ADC operation, TWI/I2C master communication, precise software timing, formatted string output, and state machine logic. All identified challenges were systematically resolved, and the system passed all laboratory tests. The recommendations outlined in Section 6.3 provide a clear roadmap for evolving this educational prototype into a fully featured, field-deployable meteorological instrument.

References

- [1] Microchip Technology Inc., "ATmega32/ATmega32L Datasheet," Document DS2503S, 2020. [Online]. Available: <https://www.microchip.com>

- [2] Aosong Electronics Co., Ltd., "DHT11 Product Manual," Version 2.0, 2020.

- [3] NXP Semiconductors, "I2C-bus specification and user manual," Rev. 7.0, UM10204, 2021.

- [4] Atmel Corporation, "AVR-libc Reference Manual," Version 2.0.0, 2022. [Online]. Available: <https://www.nongnu.org/avr-libc/>

- [5] Texas Instruments / Philips, "HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver," Datasheet, 2000.

- [6] NXP Semiconductors, "PCF8574 Remote 8-bit I/O expander for I2C-bus," Product Data Sheet, Rev. 5, 2019.

- [7] Pont, M. J., "Patterns for Time-Triggered Embedded Systems," Addison-Wesley, 2001.

- [8] Barr, M., and Massa, A., "Programming Embedded Systems," 2nd ed., O'Reilly Media, 2006.